Case Study

NETWORK TRAFFIC ANALYSIS, EMAIL FORENSICS AND PORT SCANNING

Author:

Annus Jawad

Date: Oct 14,2026

Tools:

Wireshark 4.6.4, Nmap 7.98

Kali Linux, Python3, base64, md5sum

Summary

I wanted to practice three core blue-team skills in one project: capturing and filtering network traffic, recovering evidence from a packet capture, and finding exposed services with a port scanner.

- **Wireshark:** captured 278 packets on my own Wi-Fi interface, applied display filters, inspected protocol layers and saved the capture.
- **Email forensics:** analyzed a practice capture ("Ann's Rendezvous") and recovered the sender's credentials, the recipient, the attachment, its hash and the meeting location, using only what was visible in the traffic.
- **Nmap:** scanned my own machine (no open ports) and a host whose owner explicitly permitted scanning (four open ports).

Everything here was done on my own machine, on a practice capture file, or on a target that allowed scanning.

Table of Contents:

Summary....1
Table of Contents:....3
Project Overview....4
Objectives....4
Theory....5
- Wireshark and Packet Capture....5
- Protocol Layers....5
- SMTP and Why Plaintext Matters....5
- Hashing (MD5)....5
- Port Scanning with Nmap....6
- Authorization....6

Part 1: Wireshark Live Capture and Filtering....7
- 1.1 Capturing traffic....7
- 1.2 Display filters....8
  - What I noticed....14
- 1.3 Reading one packet....15
- 1.4 Saving the capture....17

Part 2: Email Forensics Investigation....19
- 2.1 Finding the SMTP sessions....19
- 2.2 Recovering the credentials....19
- 2.3 The two emails....19
- 2.4 Extracting the attachment....20
- 2.5 Inside the document....21
- 2.6 Findings....22

Part 3: Port Scanning with Nmap....23
- 3.1 My own machine....23
- 3.2 A host that permits scanning....25

Results at a Glance....26
- Takeaways....26

Conclusion....27
References....27

Project Overview

This Project covered three connected skills used in SOC work: capturing and filtering network traffic, recovering evidence from a packet capture, and identifying open ports with a scanner.

1. In Group A, Wireshark 4.6.4 was used to capture live traffic on the wlan0 interface of my own machine. Display filters (tcp, http, icmp, dns, ip.addr, tcp.port == 80, http.request.method == "GET") were applied and the packet layers were inspected. The capture was saved as capturetesting.pcapng.
2. In Group B, a provided packet capture (Puzzle.pcap) was analyzed as a forensic exercise. SMTP sessions were followed, credentials were decoded from base64, an email attachment was extracted and hashed, and the document's embedded image was hashed.
3. In Group C, Nmap 7.98 was used to scan my own computer (127.0.0.1) and the instructor-authorized host www.douglas.stebila.ca.

All capturing and scanning was done on my own system, on a provided capture file, or on a target the assignment explicitly authorized.

Objectives

- Capture live traffic and isolate it with display filters.
- Read a packet layer by layer.
- Rebuild an SMTP email session from a capture, decode credentials, extract an attachment and verify it with a hash.
- Scan a local and an authorized remote host and interpret the output.
- Document the work so someone else could reproduce it.

Theory

Wireshark and Packet Capture

Wireshark is a packet analyzer that records frames from a network interface and decodes them by protocol. It has two kinds of filters. A capture filter is set before recording and discards unmatched traffic permanently. A display filter is applied after capture and only hides packets from view, so the data stays in the file.

Protocol Layers

Layer	Protocol (example)	Role
Link	Ethernet II	Local delivery using MAC addresses
Network	IPv4 / IPv6	Addressing and routing between hosts
Transport	TCP / UDP	Ports; TCP adds reliable ordered delivery
Application	HTTP, DNS, SMTP, TLS	The actual data exchanged

SMTP and Why Plaintext Matters

SMTP delivers email. With AUTH LOGIN, the username and password are sent as base64 strings. Base64 is reversible by anyone, so if the session is not protected by TLS, the credentials are effectively sent in cleartext. Email content and attachments (MIME, base64-encoded) can be reconstructed from a capture.

Hashing (MD5)

A hash is a fixed-length fingerprint of a file's bytes. It is used in forensics to show that a recovered file is identical to the original. The MD5 is calculated on the decoded file, not on the base64 text in the email. MD5 is acceptable for identifying files in an exercise like this, but it is no longer collision-resistant, so SHA-256 is preferred for evidence integrity.

Port Scanning with Nmap

Nmap sends probes to a host to determine which TCP/UDP ports respond.

State	Meaning
open	A service is listening and accepted the probe
closed	The host replied, but nothing is listening
filtered	No reply, usually a firewall dropping packets

Option	Purpose
-F	Fast scan: the 100 most common ports
-A	Aggressive: OS detection, version detection, scripts, traceroute
-T4	Faster timing template

Authorization

Capturing traffic and scanning ports can be illegal or against policy without permission. This task was limited to my own system, a provided capture file, and a host whose owner authorized scanning in the assignment.

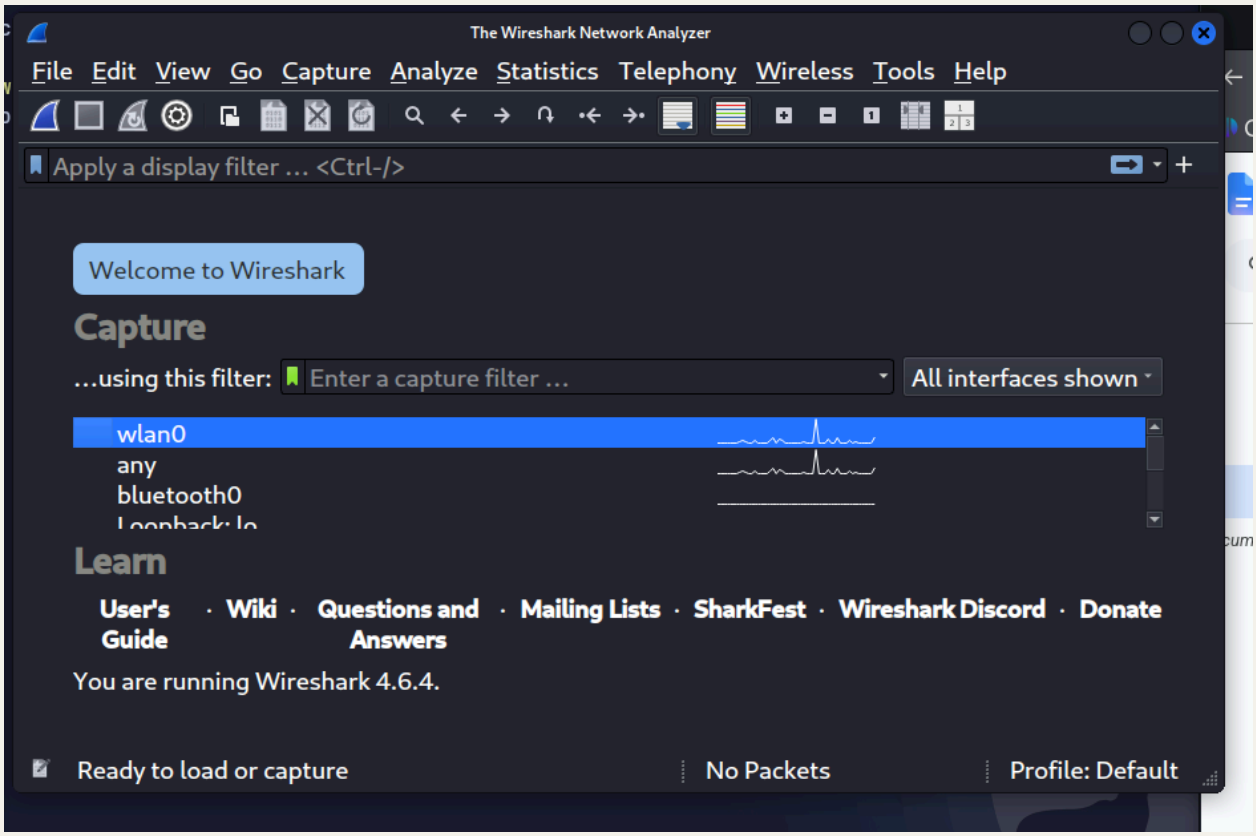
Part 1: Wireshark Live Capture and Filtering

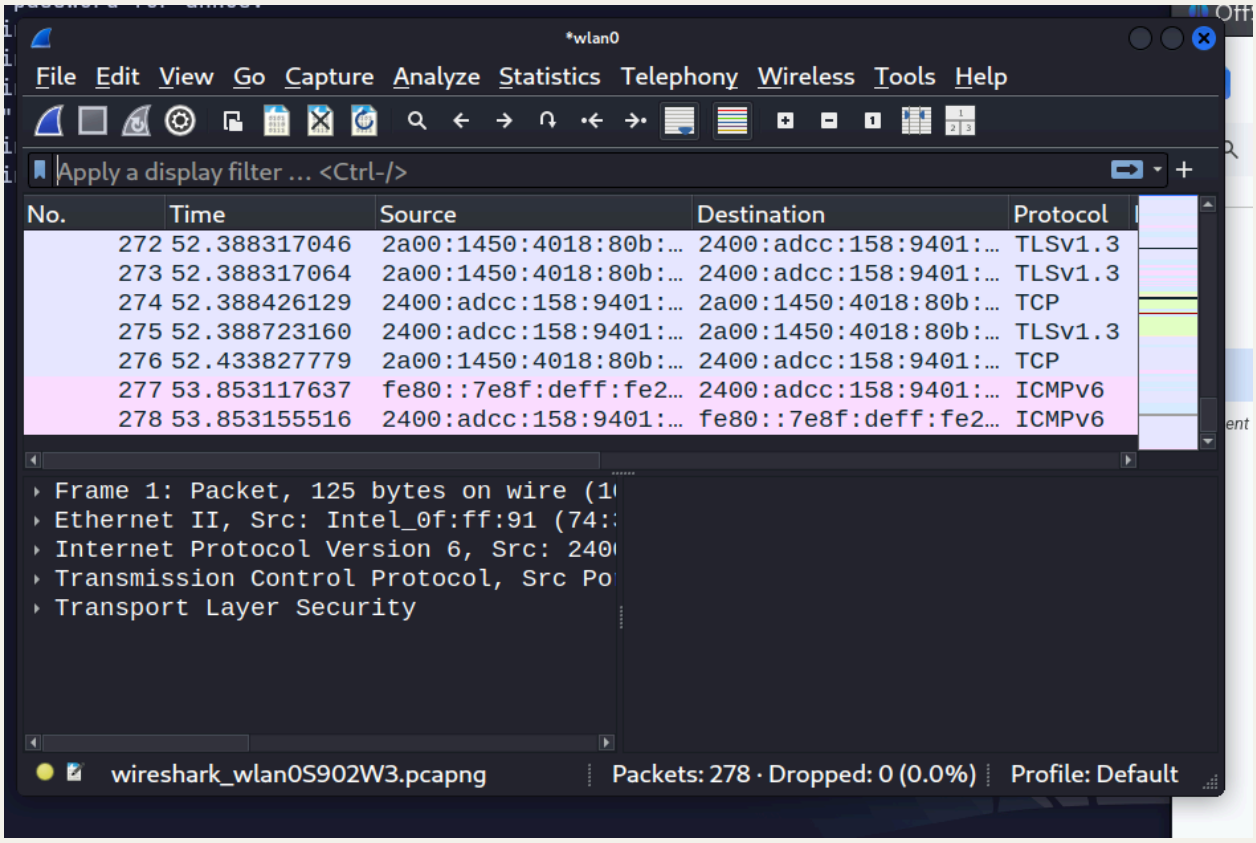
Setup: Wireshark 4.6.4 on Kali Linux, interface wlan0, local IP 192.168.1.205, gateway and DNS server 192.168.1.1. Captured on October 14, 2026.

1.1 Capturing traffic

I selected wlan0 (it showed live activity), started the capture, generated some traffic and stopped it. The result was 278 packets with 0 dropped.

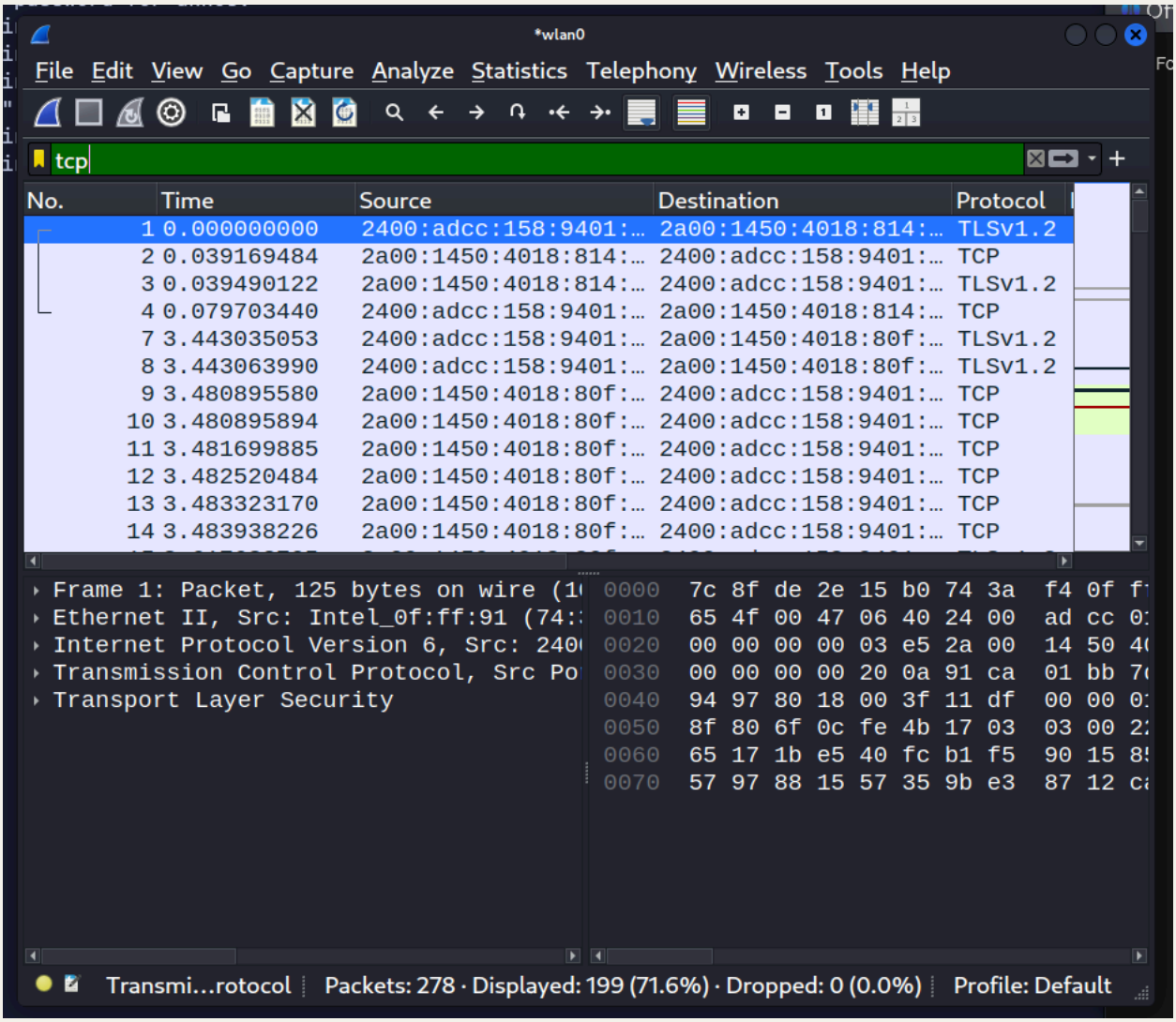
Wireshark welcome screen with interfaces Live capture on wlan0



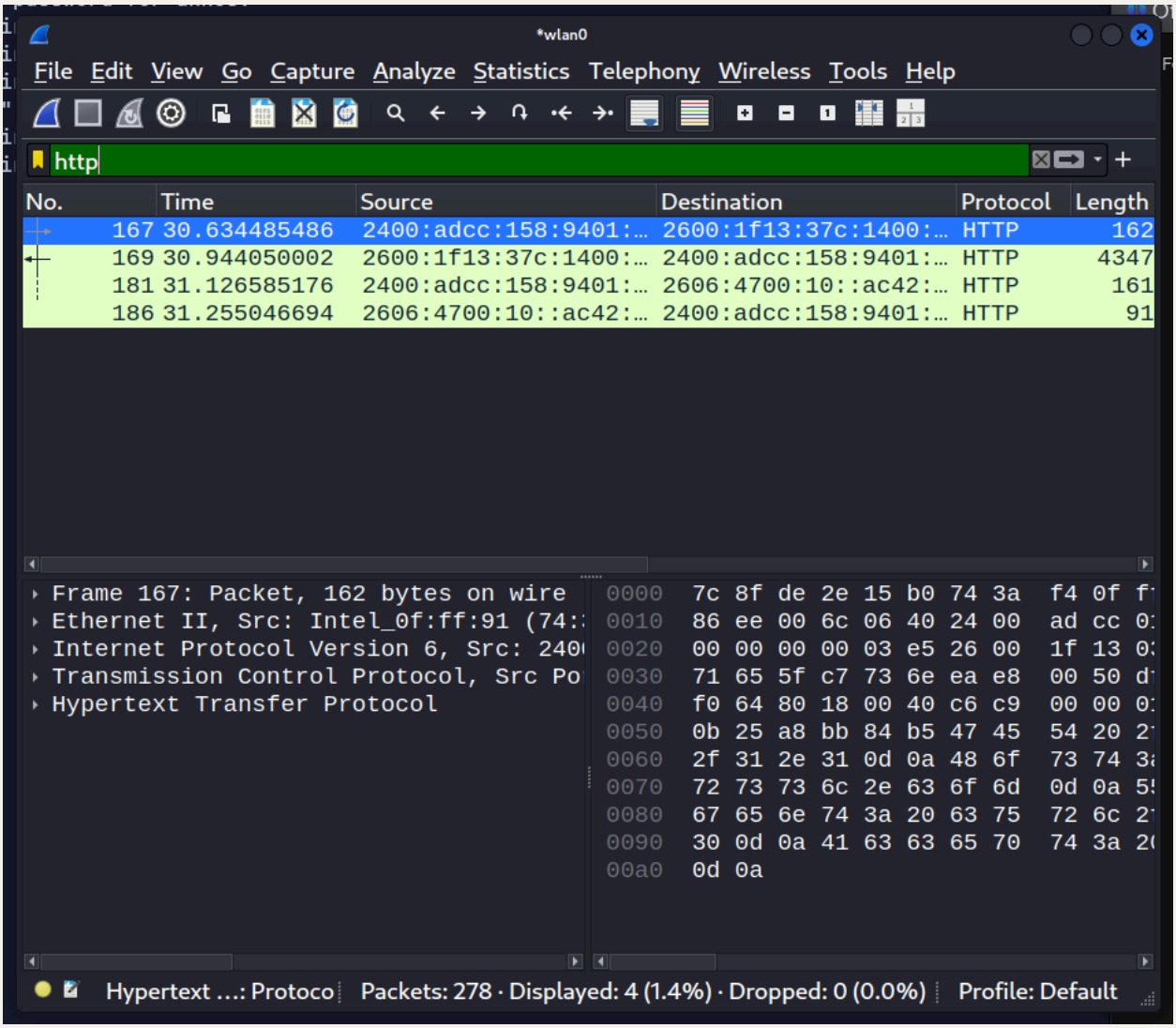


1.2 Display filters

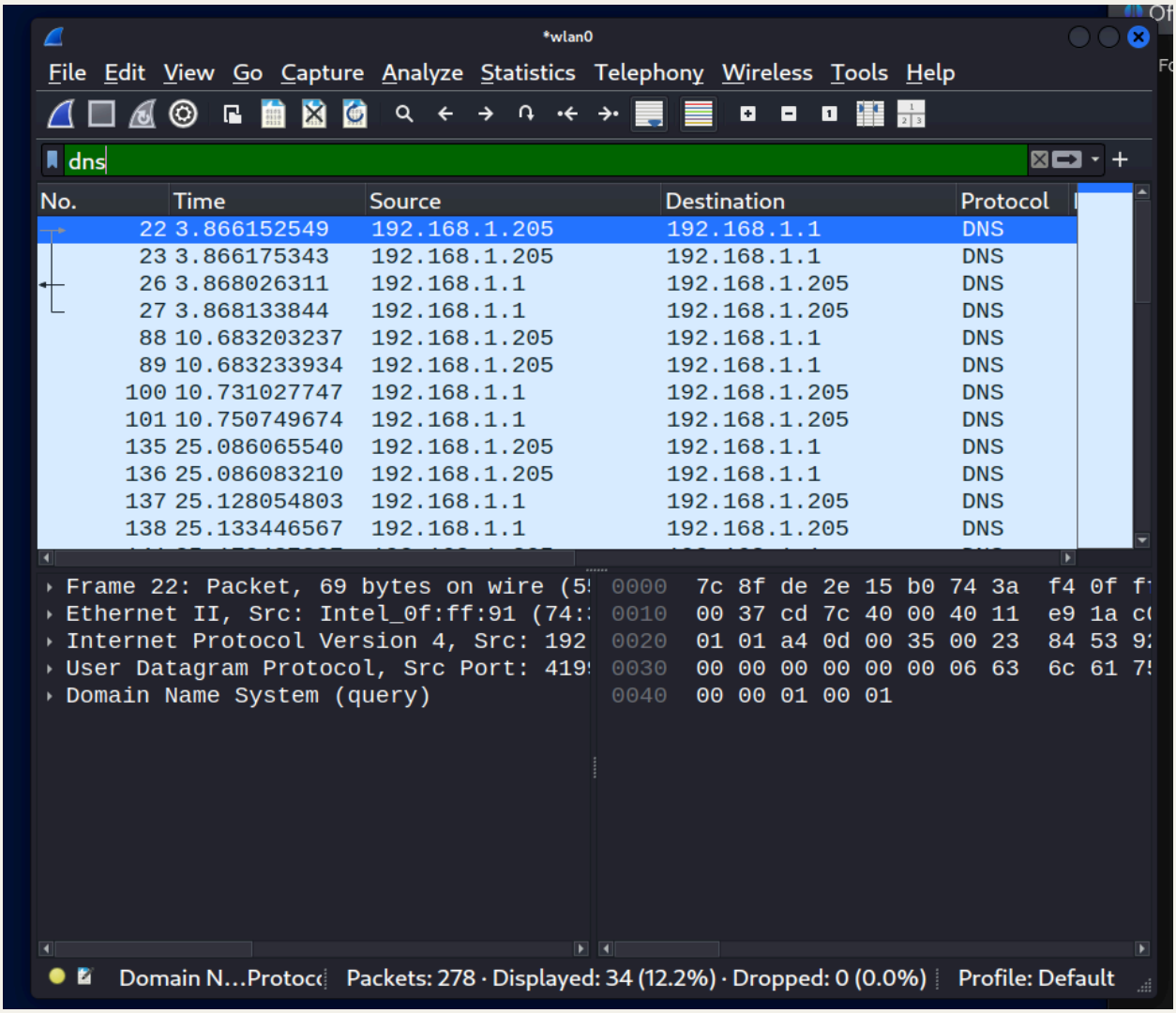
Filter	Shows	Packets displayed (of 278)
tcp	All TCP traffic	199 (71.6%)
http	Unencrypted HTTP	4 (1.4%)
dns	Name lookups	34 (12.2%)
ip.addr == 192.168.1.205	IPv4 traffic to/from my host	43 (15.5%)
tcp.port == 80	Port 80 traffic	27 (9.7%)
http.request.method == "GET"	HTTP GET requests	2 (0.7%)


*wlan0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp
No. Time Source Destination Protocol
1 0.000000000 2400:adcc:158:9401:… 2a00:1450:4018:814:… TLSv1.2
2 0.039169484 2a00:1450:4018:814:… 2400:adcc:158:9401:… TCP
3 0.039490122 2a00:1450:4018:814:… 2400:adcc:158:9401:… TLSv1.2
4 0.079703440 2400:adcc:158:9401:… 2a00:1450:4018:814:… TCP
7 3.443035053 2400:adcc:158:9401:… 2a00:1450:4018:80f:… TLSv1.2
8 3.443063990 2400:adcc:158:9401:… 2a00:1450:4018:80f:… TLSv1.2
9 3.480895580 2a00:1450:4018:80f:… 2400:adcc:158:9401:… TCP
10 3.480895894 2a00:1450:4018:80f:… 2400:adcc:158:9401:… TCP
11 3.481699885 2a00:1450:4018:80f:… 2400:adcc:158:9401:… TCP
12 3.482520484 2a00:1450:4018:80f:… 2400:adcc:158:9401:… TCP
13 3.483323170 2a00:1450:4018:80f:… 2400:adcc:158:9401:… TCP
14 3.483938226 2a00:1450:4018:80f:… 2400:adcc:158:9401:… TCP
Frame 1: Packet, 125 bytes on wire (1
Ethernet II, Src: Intel_0f:ff:91 (74:
Internet Protocol Version 6, Src: 240
Transmission Control Protocol, Src Po
Transport Layer Security
0000 7c 8f de 2e 15 b0 74 3a f4 0f f
0010 65 4f 00 47 06 40 24 00 ad cc 0
0020 00 00 00 00 03 e5 2a 00 14 50 4
0030 00 00 00 00 20 0a 91 ca 01 bb 7
0040 94 97 80 18 00 3f 11 df 00 00 0
0050 8f 80 6f 0c fe 4b 17 03 03 00 2
0060 65 17 1b e5 40 fc b1 f5 90 15 8
0070 57 97 88 15 57 35 9b e3 87 12 c
Transmi…rotocol
Packets: 278 · Displayed: 199 (71.6%) · Dropped: 0 (0.0%)
Profile: Default

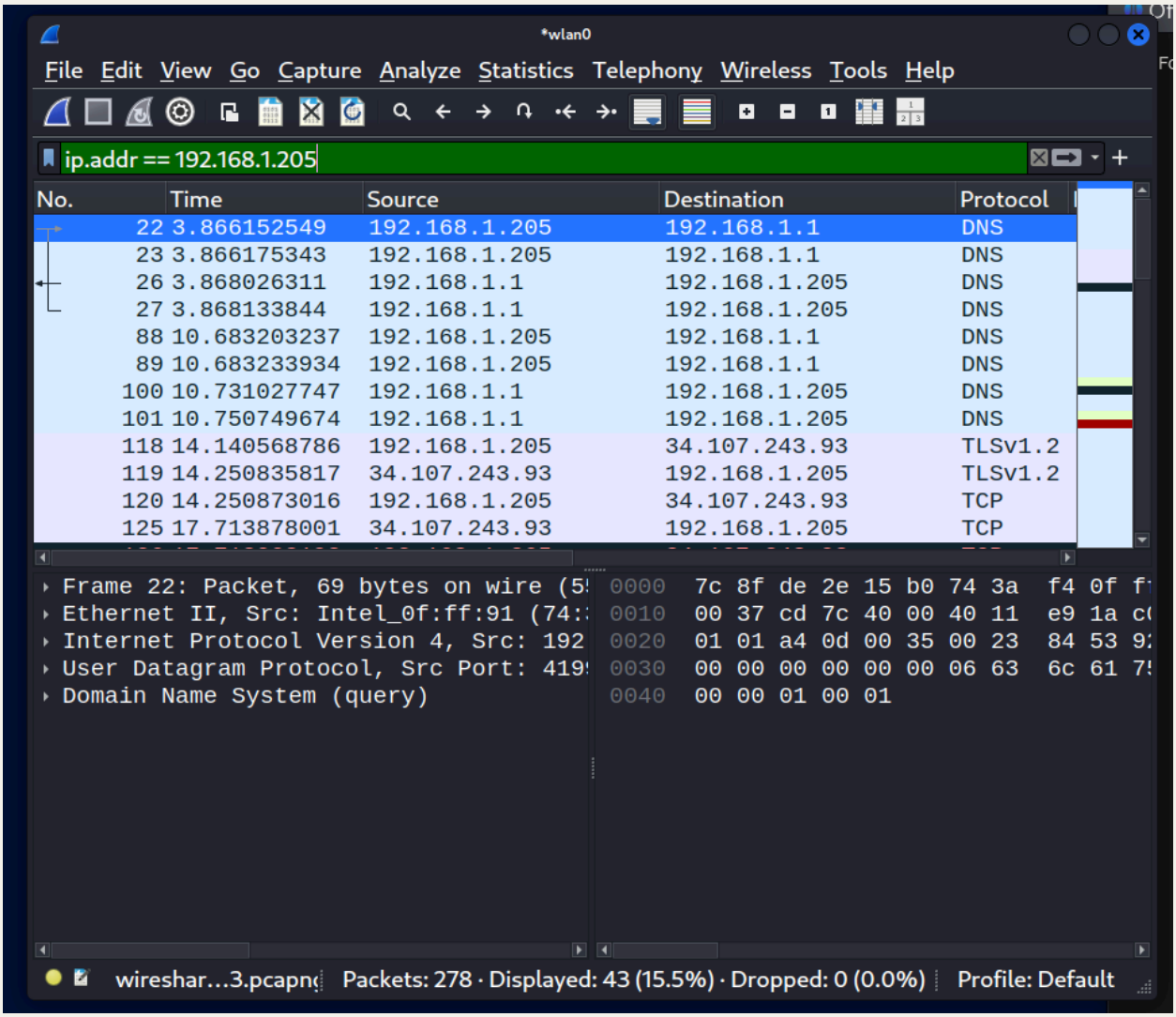
tcp filter


*wlan0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http
No. Time Source Destination Protocol Length
167 30.634485486 2400:adcc:158:9401:… 2600:1f13:37c:1400:… HTTP 162
169 30.944050002 2600:1f13:37c:1400:… 2400:adcc:158:9401:… HTTP 4347
181 31.126585176 2400:adcc:158:9401:… 2606:4700:10::ac42:… HTTP 161
186 31.255046694 2606:4700:10::ac42:… 2400:adcc:158:9401:… HTTP 91
Frame 167: Packet, 162 bytes on wire
Ethernet II, Src: Intel_0f:ff:91 (74:
Internet Protocol Version 6, Src: 240
Transmission Control Protocol, Src Po
Hypertext Transfer Protocol
0000 7c 8f de 2e 15 b0 74 3a f4 0f f
0010 86 ee 00 6c 06 40 24 00 ad cc 0
0020 00 00 00 00 03 e5 26 00 1f 13 0
0030 71 65 5f c7 73 6e ea e8 00 50 d
0040 f0 64 80 18 00 40 c6 c9 00 00 0
0050 0b 25 a8 bb 84 b5 47 45 54 20 2
0060 2f 31 2e 31 0d 0a 48 6f 73 74 3
0070 72 73 73 6c 2e 63 6f 6d 0d 0a 5
0080 67 65 6e 74 3a 20 63 75 72 6c 2
0090 30 0d 0a 41 63 63 65 70 74 3a 2
00a0 0d 0a
Hypertext …: Protoco
Packets: 278 · Displayed: 4 (1.4%) · Dropped: 0 (0.0%)
Profile: Default

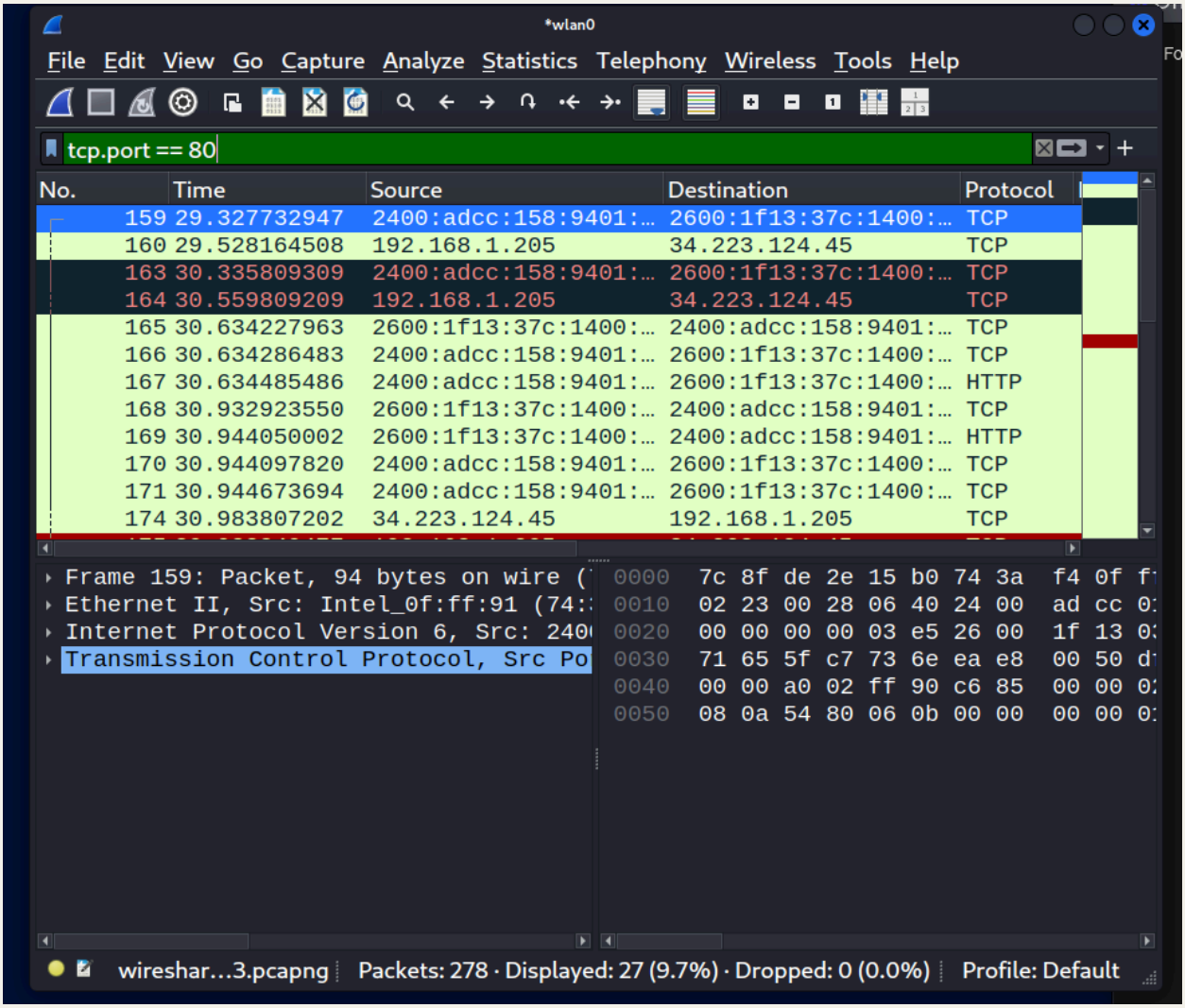
http filter


*wlan0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
dns
No. Time Source Destination Protocol
22 3.866152549 192.168.1.205 192.168.1.1 DNS
23 3.866175343 192.168.1.205 192.168.1.1 DNS
26 3.868026311 192.168.1.1 192.168.1.205 DNS
27 3.868133844 192.168.1.1 192.168.1.205 DNS
88 10.683203237 192.168.1.205 192.168.1.1 DNS
89 10.683233934 192.168.1.205 192.168.1.1 DNS
100 10.731027747 192.168.1.1 192.168.1.205 DNS
101 10.750749674 192.168.1.1 192.168.1.205 DNS
135 25.086065540 192.168.1.205 192.168.1.1 DNS
136 25.086083210 192.168.1.205 192.168.1.1 DNS
137 25.128054803 192.168.1.1 192.168.1.205 DNS
138 25.133446567 192.168.1.1 192.168.1.205 DNS
Frame 22: Packet, 69 bytes on wire (5
Ethernet II, Src: Intel_0f:ff:91 (74:
Internet Protocol Version 4, Src: 192
User Datagram Protocol, Src Port: 419
Domain Name System (query)
0000 7c 8f de 2e 15 b0 74 3a f4 0f f
0010 00 37 cd 7c 40 00 40 11 e9 1a c
0020 01 01 a4 0d 00 35 00 23 84 53 9
0030 00 00 00 00 00 00 06 63 6c 61 7
0040 00 00 01 00 01
Domain N...Protoc
Packets: 278 · Displayed: 34 (12.2%) · Dropped: 0 (0.0%)
Profile: Default

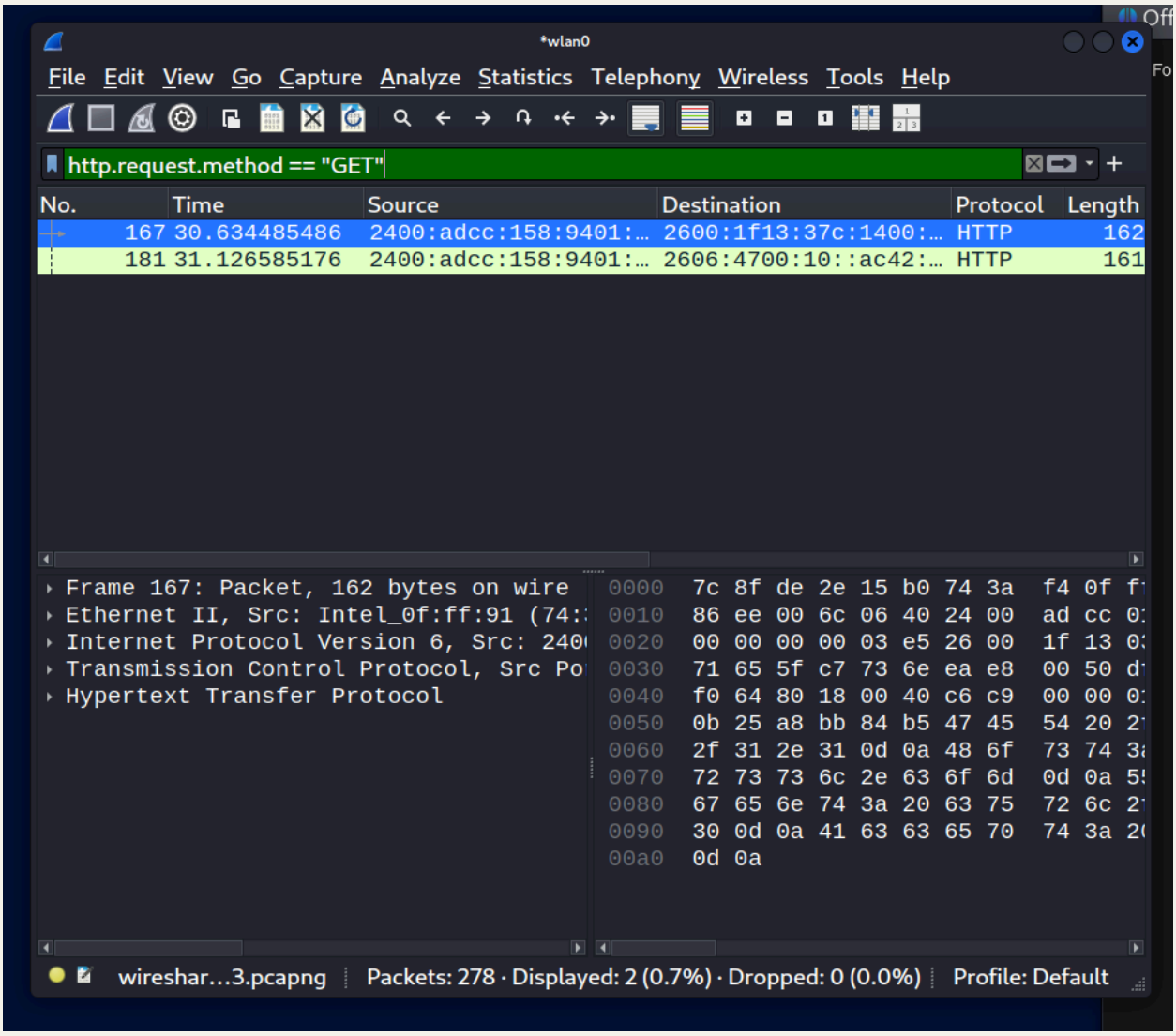
dns filter



ip.addr filter



tcp.port == 80 filter



GET filter

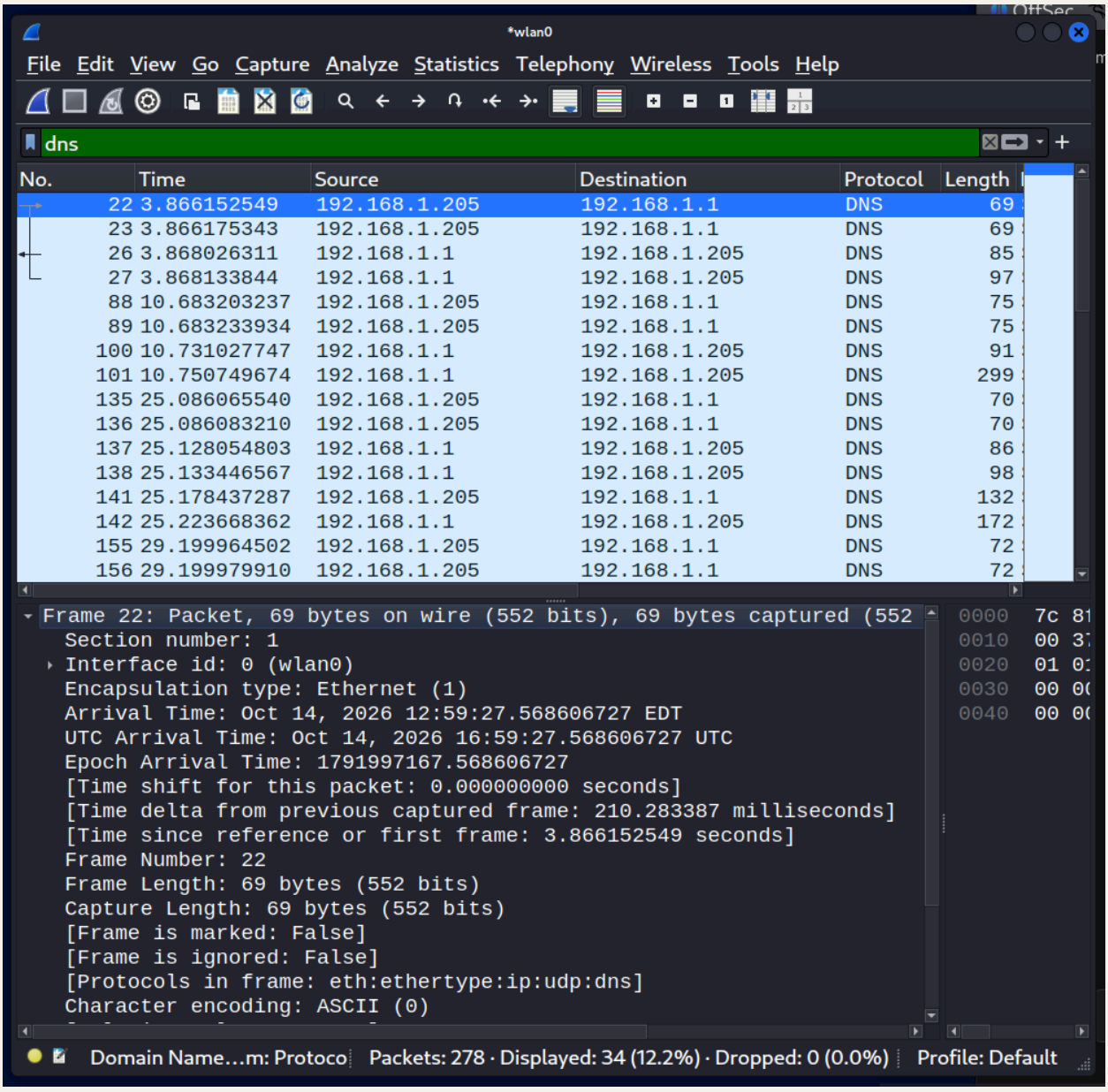
What I noticed

- Most TCP traffic was TLS (1.2 and 1.3), so only metadata is visible, not content.
- Only 4 HTTP packets were captured (2 requests and 2 responses). Almost everything else was encrypted.
- DNS traffic went between my host and the router, which acts as the resolver.
- A lot of the traffic was IPv6, so the IPv4 filter ip.addr only matched 43 packets. An IPv6 host filter uses ipv6.addr.
- I did not capture an icmp filter result, though ICMPv6 packets (neighbor discovery) were visible in the list.

1.3 Reading one packet

I expanded Frame 22, a DNS query:

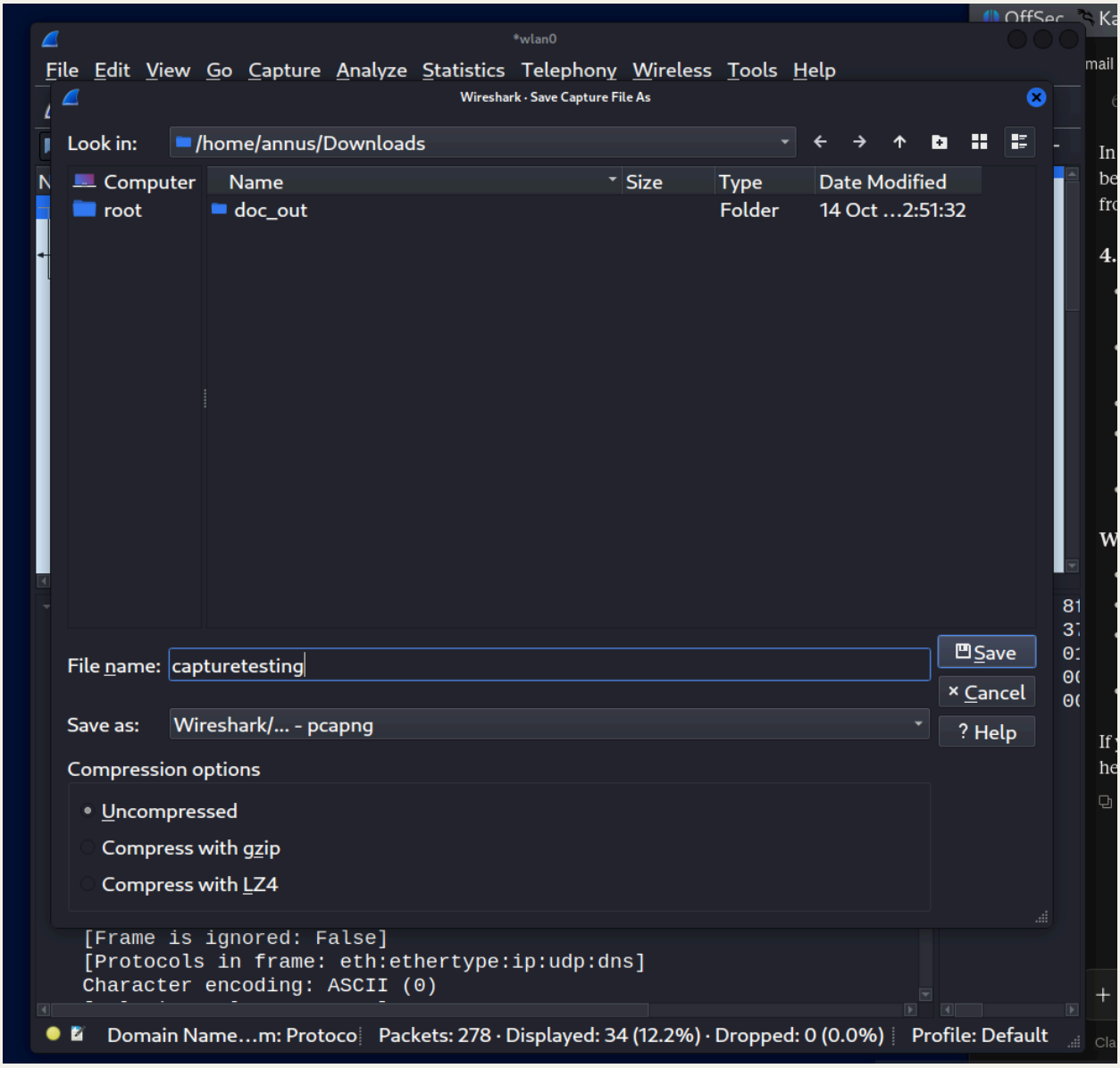
Field	Value
Layers	Frame, Ethernet II, IPv4, UDP, DNS (query)
Frame length	69 bytes (552 bits)
Interface	wlan0
Protocols in frame	eth:ethertype:ip:udp:dns
Arrival time	Oct 14, 2026 12:59:27 EDT
Source -> destination	192.168.1.205 -> 192.168.1.1



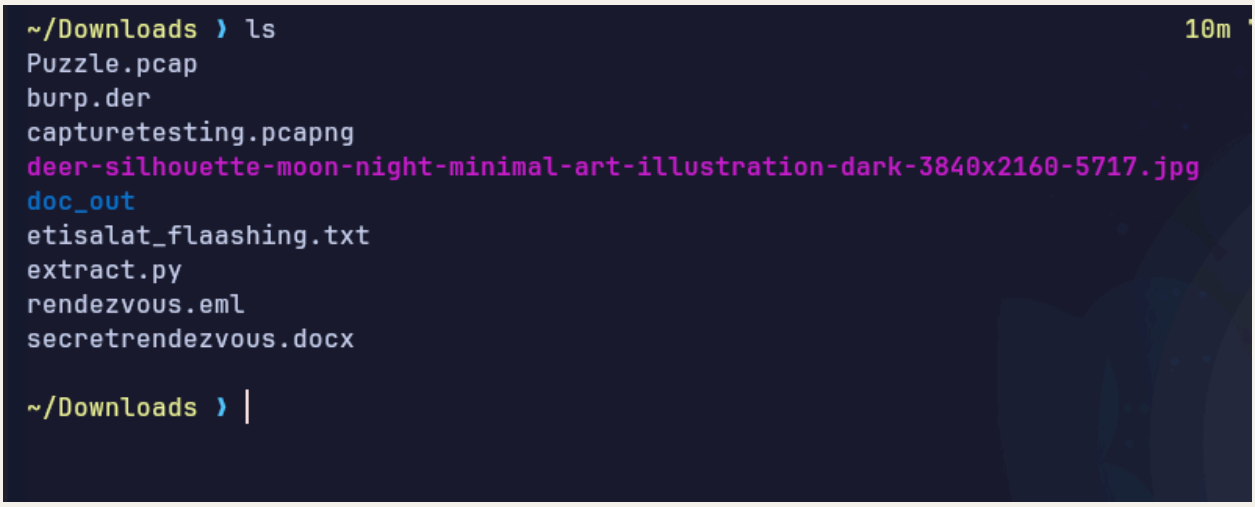
Frame 22 expanded

1.4 Saving the capture

Saved with File -> Save As as capturetesting.pcapng (pcapng, uncompressed).



Save dialog



Saved file listing

Part 2: Email Forensics Investigation

Scenario: From a practice packet capture ("Ann's Rendezvous"), work out what Ann emailed, where she went, and recover the evidence.

2.1 Finding the SMTP sessions

I filtered for SMTP and used Follow -> TCP Stream on each session. There were two sessions to AOL mail servers, both from a host named annlaptop on 10 Oct 2009.

2.2 Recovering the credentials

Both sessions used AUTH LOGIN with no TLS. Decoding the base64:

```
echo 'c25lYWt5ZzMza0Bhb2wuY29t' | base64 -d    ->  sneakyg33k@aol.com
echo 'NTU4cjAwbHo='            | base64 -d    ->  558r00lz
```

The server replied 235 AUTHENTICATION SUCCESSFUL. (These are fictional credentials from a public practice scenario.)

2.3 The two emails

	Email 1	Email 2
Recipient	sec558@gmail.com	mistersecretx@aol.com
Subject	lunch next week	rendezvous
Body	Cancels a lunch, "heading out of town"	"Bring your fake passport and a bathing suit. Address attached."
Attachment	None	secretrendezvous.docx
Relevance	Unrelated	The target email

2.4 Extracting the attachment

I saved the second email from the capture (File -> Export Objects -> IMF) as rendezvous.eml and extracted the attachment with Python. My first script was named email.py, which shadowed Python's built-in email module and broke the import, so I renamed it extract.py.

```
import email
from email import policy


msg = email.message_from_file(open("rendezvous.eml"), policy=policy.default)
for part in msg.iter_attachments():
    name = part.get_filename()
    open(name, "wb").write(part.get_payload(decode=True))
    print("saved", name)
```

```
python3 extract.py
md5sum secretrendezvous.docx
```

```
9e423e11db88f01bbff81172839e1923 secretrendezvous.docx
```

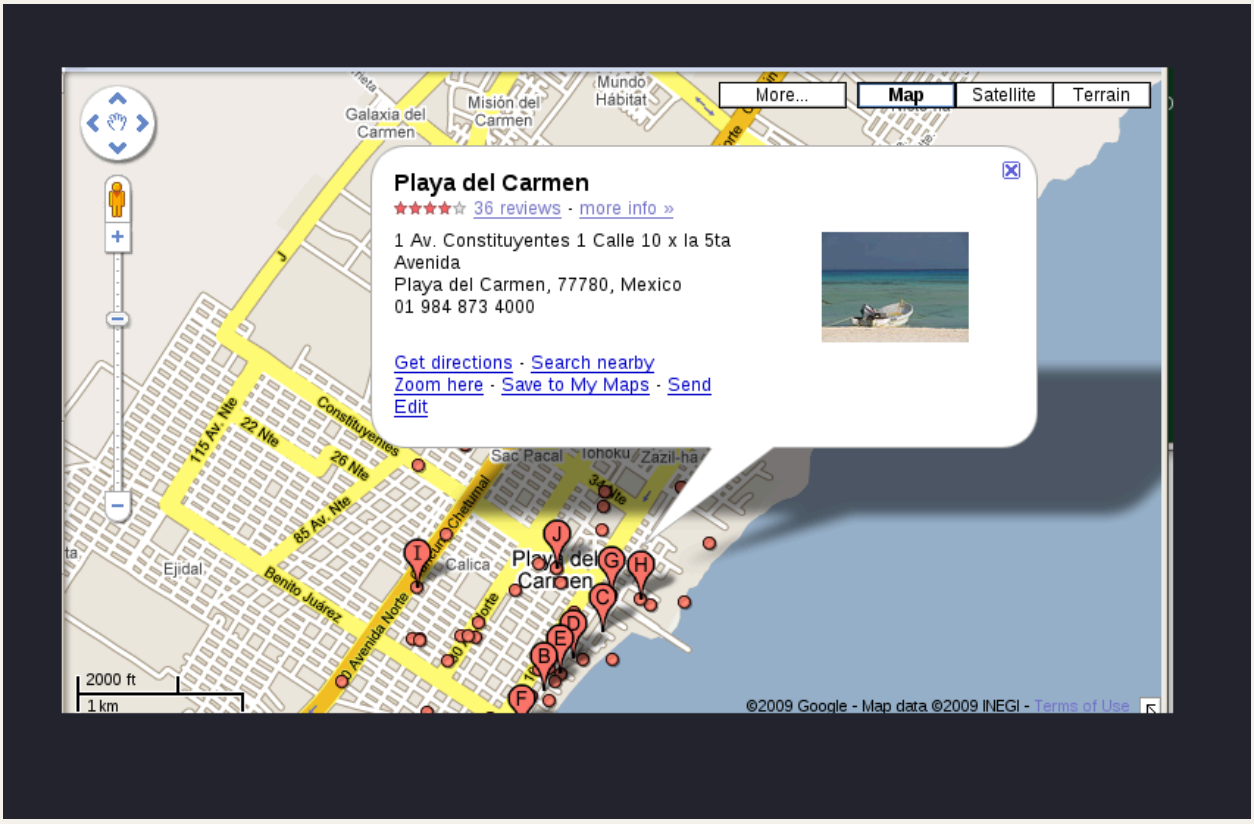
2.5 Inside the document

A .docx is a zip archive, so I unzipped it:

```
unzip secretrendezvous.docx -d doc_out
md5sum doc_out/word/media/*
```

The only media file was image1.png: aadeace50997b1ba24b09ac2ef1940b7

The image is a Google Maps view with a marker at an address in Playa del Carmen, Mexico.



Map image from the document

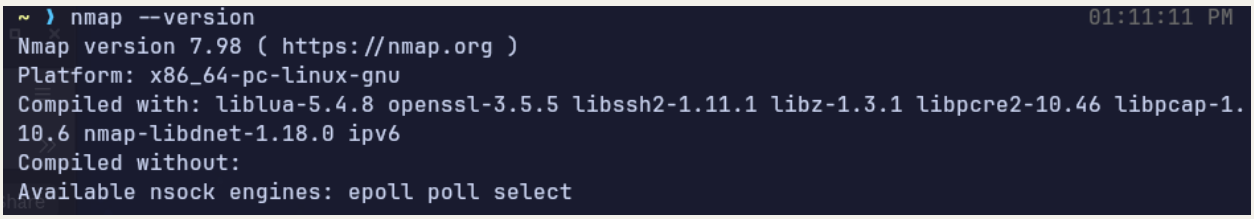
2.6 Findings

Question	Answer
Sender's email	sneakyg33k@aol.com
Sender's password	558r00lz
Recipient's email	mistersecretx@aol.com
Items to bring	Fake passport, bathing suit
Attachment name	secretrendezvous.docx
MD5 of attachment	9e423e11db88f01bbff81172839e1923
Meeting location	Playa del Carmen, Mexico
MD5 of embedded image	aadeace50997b1ba24b09ac2ef1940b7

`Evidence chain:` plaintext SMTP in the capture -> credentials and recipient -> attachment extracted and hashed -> embedded image -> location**`.`**

Part 3: Port Scanning with Nmap

Nmap 7.98 on x86_64 Linux.



nmap --version

3.1 My own machine

```
nmap -A -T4 127.0.0.1
```

- Host up, 0.00017s latency.
- All 1000 scanned ports were closed (reset), so no services were listening.
- OS detection was inconclusive (too many fingerprints matched).
- Completed in 1.73 seconds.



Localhost scan

Nothing unexpected was running. If a service had been found that I did not need, I would stop and disable it, then re-scan to confirm:

```
sudo systemctl stop <service>
sudo systemctl disable <service>
```

3.2 A host that permits scanning

```
nmap -F www.douglas.stebila.ca
```

I had the permission to scan it in the exercise I was following.

Port	State	Service	Purpose
22/tcp	open	ssh	Encrypted remote administration
80/tcp	open	http	Unencrypted web traffic
443/tcp	open	https	Encrypted web traffic (TLS)
8443/tcp	open	https-alt	Alternate HTTPS port, often a second web service or admin interface

The host resolves to an Amazon EC2 address. 96 other ports showed as filtered (no response, likely firewalled). The scan took 39.19 seconds.

```
~ ❱ nmap -F www.douglas.stebila.ca                                    01:12:16 PM
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-14 13:14 -0400
Nmap scan report for www.douglas.stebila.ca (54.175.72.151)
Host is up (0.59s latency).
rDNS record for 54.175.72.151: ec2-54-175-72-151.compute-1.amazonaws.com
Not shown: 96 filtered tcp ports (no-response)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
443/tcp  open  https
8443/tcp open  https-alt

Nmap done: 1 IP address (1 host up) scanned in 39.19 seconds
```

Remote scan

I used -F (fast scan) instead of -A to keep the scan light.

Results at a Glance

Item	Value
Packets captured	278 (0 dropped)
HTTP packets	4 of 278 (1.4%)
TCP share of traffic	71.6%
Localhost scan	1000 closed ports, no services
Remote host open ports	22, 80, 443, 8443
Attachment MD5	9e423e11db88f01bbff81172839e1923
Embedded image MD5	aadeace50997b1ba24b09ac2ef1940b7

Takeaways

1. Filters make big captures manageable. 278 packets became 4 HTTP packets with one filter.
2. Encryption changes what an analyst can see. TLS traffic shows endpoints and timing but not content. The plaintext SMTP session exposed the login, the message and the attachment.
3. Base64 is not security. Anyone on the path can decode it. Use TLS for email.
4. Hashes prove integrity. Hash the decoded file, and also hash the original capture so the chain of evidence can be checked.
5. Open ports are attack surface. My machine exposed nothing. The remote host exposed SSH and web services, which is normal for a server but needs hardening (key-only SSH, patching, firewall rules).
6. Small mistakes are worth documenting. Naming a script email.py broke my import. File names can shadow standard libraries.

Conclusion

This Project showed how three common SOC activities fit together. Wireshark's display filters make a large capture manageable (for example, 278 packets reduced to 4 HTTP packets), and layer-by-layer inspection explains how a single packet is built. The forensic exercise showed how much can be recovered from an unencrypted SMTP session: the sender's login, the recipient, the message text and the attachment, all of which could be reconstructed and verified with a hash. The scans showed my own system exposing no listening services, while the authorized remote host exposed SSH and web services, which illustrates how open ports define an attack surface.

Security lessons: use TLS for email and web traffic, since base64 is not protection; close unneeded ports; verify evidence with hashes; and only capture or scan with authorization. Everything in this task was done on my own machine, on a provided capture file, or on a target the assignment authorized.

References

- Wireshark User's Guide and Display Filter Reference. https://www.wireshark.org/docs/
- Nmap Reference Guide. https://nmap.org/book/man.html
- Nmap: Port Scanning Basics. https://nmap.org/book/port-scanning.html
- RFC 5321, Simple Mail Transfer Protocol. https://www.rfc-editor.org/rfc/rfc5321
- RFC 4954, SMTP Service Extension for Authentication. https://www.rfc-editor.org/rfc/rfc4954
- Python email package. https://docs.python.org/3/library/email.html
- SpeedGuide ports database. http://www.speedguide.net/ports.php